LINKED LIST :-

Why there is a need of linked list ?

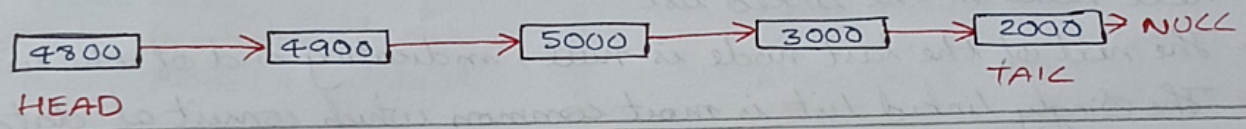
If we declare an array of size 3. As we know that all the values of an array are stored in a continous manner, so all 3 values of an array are stored in a sequential fashion.
Then, total memory space occupied by array would be 3*4=12 bytes

Draw backs of using array :-

* We cannot insert more than 3 elements in above example, because only 3 spaces are allocated by 3 elements.
* In case of array, the wastage of memory can occur.
* In array, we are providing fixed-size at compile time, due to which wastage of memory occurs. The solution to this problem is to use linked list.

What is Linked List ?

* A Linked List is also a collection of elements, but the elements are not stored in a consecutive location.
* Linked List is a collection of the nodes in which one node is connected to another node and node consists of two parts, one is data part and second one is the address part.



NOTE :-

* Linked Lists were invented to overcome some of the limitations of arrays, especially in cases where you need :
  * Frequent Insertions / deletions
  * Dynamic Size
  * No need for random access
* Nodes are created as needed → no fixed size

* Linked List is a part of the Collection framework present in java.util Package.
* This class is an implementation of the Linked List data structure, which is a linear data structure where the elements are not stored in contiguous locations, and every element is a seperate object with a data part and an address part.
* The elements are linked using pointers and addresses, and each element is known as a node.

NOTE :-

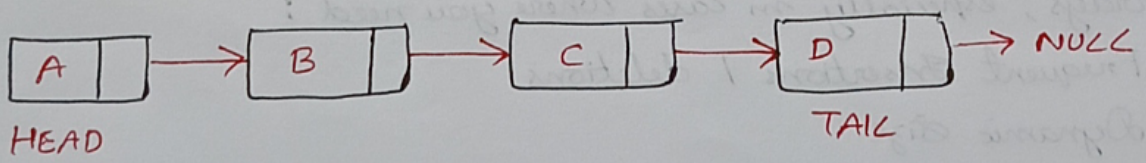
* The nodes cannot be accessed directly instead we have to start from the head and follow the link until we find the node that we want.

* Since a LinkedList acts as a dynamic array and we do not have to specify the size while creating it, the size of the list automatically increases when we dynamically add & remove items.

Types of Linked List :-

① Singly Linked List :-

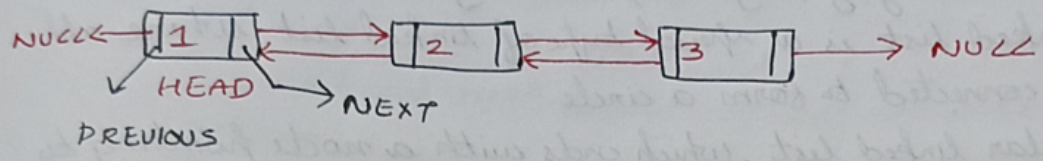
* A Singly linked list is a fundamental data structure, it consists of nodes where each contain a data field and a reference to the next node in the linked list.
* The next of the last node is null, indicating end of the list.
* The Singly linked list is most common which consist of data part and address part.
* The address part in the node is known as a pointer.



* Null means its address part does not point to any node. The pointer that holds the address of the initial node is known as a head pointer.

② Doubly Linked List :-

* A doubly linked list is a more complex data structure than a singly linked list, but it offers several advantages.
* The main advantage of a doubly linked list is that it allows for efficient traversal of the list in the both directions.
* This is because each node in the list contains a pointer to the previous node and a pointer to the next node.
* This allows for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of list in both directions



* As the name suggests, the doubly linked list contains two pointers. We define it in three parts, the data part (1) and the two address part (previous, next)

Nodes :-

SINGLE LINKED LIST :-

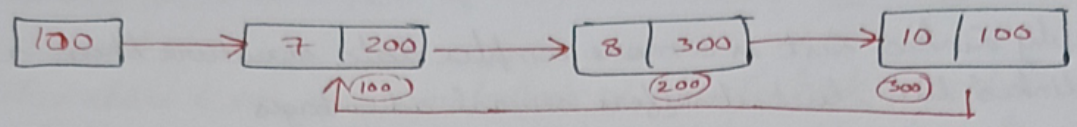
```
public class Node {
    int data;
    Node next;
// Constructor to intialize the node
    public Node(int data){
        this.data = data;
        this.next = null;
    }
}
```

DOUBLY LINKED LIST :-

```
class Node {
    // To store the value or data
    int data;
    // Reference to previous node
    Node prev;
    // Reference to next node
    Node next;
    // Constructor
    Node(int d){
        data = d;
        prev = next = null;
    }
};
```

③ Circular Linked List :-

* A circular linked list is a variation of a singly linked list.
* The only difference is " last node does not point to any node in a singly linked list.



* A circular linked list is a data structure where the last node connects back to the first, forming a loop.
* This structure allows for continous traversal without any interruptions.
* Circular linked lists are especially helpful for tasks like scheduling and managing playlists, allowing for smooth navigation.
* A circular linked list is a special type of linked list where all the nodes are connected to form a circle.
* Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node.

Q1) Implement a Singly Linked List in Java **

```
class Node {
    int data;
    Node next;
    Node(int d) {
        data = d;
    }
}
public class SinglyLinkedList {
    Node head;
public void insertAtEnd(int data){
    Node newNode = new Node(data);
    if(head==null) {
        head = newNode;
        return;
    }
    Node curr = head;
    while(curr != null) {
        System.out.print(curr.data + " -> ");
        curr = curr.next; } System.out.println("null"); } }
```

Q2) Insert a node at the beginning of a Singly Linked List

```
public void insertAtBeginning(int data){
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
}
```

Q3) Delete a node from a Singly Linked List *

```
public void deleteNode(int key){
        if(head == null)
        return;
        if(head.data == key){
        head = head.next;
        return;
        }
        Node curr = head;
        while(curr.next != null && curr.next.data != key)
                curr = curr.next;
        if(curr.next != null)
                curr.next = curr.next.next;
}
```

Q4) Implement a Doubly Linked List in Java *

```
class DNode {
    int data;
    DNode prev, next;
    DNode(int d){
        data = d;
    }
}
public class DoublyLinkedList{
    DNode head, tail;
public void insertAtEnd(int data){
    DNode newNode = new DNode(data);
    if(head == null){
        head = tail = newNode;
        return; }
    tail.next = newNode;
    newNode.prev = tail;
    tail = newNode;
}
public void printForward(){
for(DNode curr = head;
    curr != null; curr = curr.next)
System.out.print(curr.data
        + " <->");
    System.out.println("null");
}
public void printBackward(){
    " "
```

Q5) Implement a Circular Singly Linked List in Java *

```
public class CircularSinglyLinkedList {
    Node head;
    public void insert(int data) {
        Node newNode = new Node(data);
        if(head == null) {
            head = newNode;
            newNode.next = head;
            return;
        }
        Node curr = head;
        while(curr.next != head) curr = curr.next;
        curr.next = newNode;
        newNode.next = head;
    }
    public void printList() {
        if(head == null) return;
        Node curr = head;
        do {
            System.out.print(curr.data + " ->");
            curr = curr.next;
        } while(curr != head);
        System.out.println("(back to head)");
    }
}
```

Q6) Implement a Circular Doubly Linked List in Java *

```
public class CircularDoublyLinkedList {
    DNode head;
    public void insert(int data) {
        DNode newNode = new DNode(data);
        if(head == null) {
            head = newNode;
            head.next = head.prev = head;
            return;
        }
        DNode tail = head.prev;
        tail.next = newNode;
```

```
        newNode.prev = tail;
        newNode.next = head;
        head.prev = newNode;
    }
    public void printForward(int steps){
        if(head == null) return;
        DNode curr = head;
        for(int i=0; i<steps; i++){
            System.out.print(curr.data + "<->");
            curr = curr.next;
        }
        System.out.println("...");
    }
}
```

07) Reverse a Linked List ***

08) Middle of Linked List *

09) Merge Two Sorted Linked List ****

010) Check if Linked List is Palindrome *

011) Detect Cycle in Linked List * *****

012) Remove Cycle in Linked List * ***

013) Flatten a Multilevel Doubly Linked List

014) Clone Linked List with Random Pointers

015) Add Two numbers *

016) LRU Cache

017) Rotate a Linked List *

018) Reverse Nodes in K-Group

019) Intersection point of two Linked List **

LINKED LIST CYCLE ****....

MERGE TWO SORTED ***

020) Reorder Linked List *

ANS IN GITHUB REPO AJNABH-KOUSHIK